



预训练语言模型进阶

目录

CONTENTS

1

模型优化

2

长文本处理

3

模型蒸馏与压缩

4

生成模型

目录

CONTENTS

1

模型优化

2

长文本处理

3

模型蒸馏与压缩

4

生成模型

□XLNet

□RoBERTa

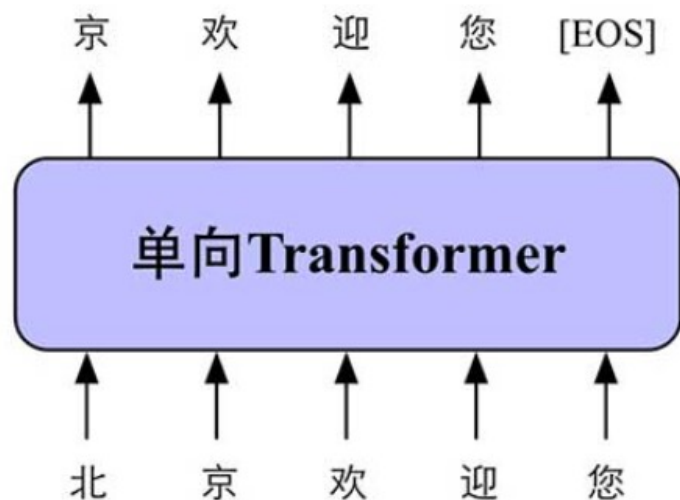
□ALBERT

□ELECTRA

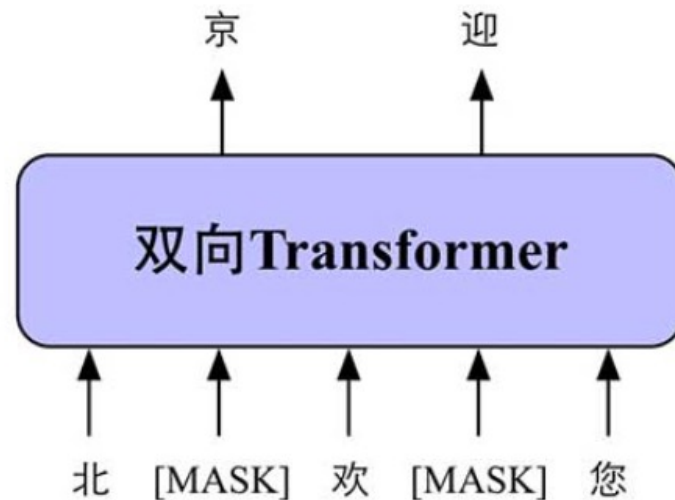
- 随着以GPT、BERT为代表的预训练语言模型的提出，很多的工作集中在进一步优化预训练语言模型，使之在各类自然语言处理任务上获得更好的效果。
- 本节将围绕模型优化方面介绍五个有代表性的预训练语言模型：XLNet、RoBERTa、ALBERT、ELECTRA和MacBERT，主要介绍模型的设计思路及具体的建模方法。

模型优化

- 模型优化——XLNet
- 语言模型通常被分为两大类：
 - 自回归语言模型
 - 自编码语言模型



(a) 自回归语言模型



(b) 自编码语言模型

□ 自回归语言模型

- 该类模型需要基于给定的历史文本序列预测下一个单词的概率，其建模方法如下所示

$$\log P(x) = \sum_{i=1}^N \log P(x_i | x_{1:i-1})$$

- 式中， x 表示整个句子； $x_{1:i-1}$ 表示下标小于 i 的历史序列，即 $x_1 \cdots x_{i-1}$

□ 自编码语言模型

□ 自编码语言模型：目标是通过上下文重构被掩码的单

□ BERT就属于这个类别，其建模公式表示为

$$\log P(x|\hat{x}) = \sum_{i=1}^N m_i \log P(x_i|\hat{x})$$

□ x 表示整个句子； $\hat{x}$ 表示经过掩码的句子； m_i 表示第 i 个词是否被掩码。

模型优化——XLNet(1/13)

❑ **XLNet**: 基于Transformer-**XL Net**，并集成了自编码语言模型的优点，主要特点包括：

- ❑ 提出了一种可以**捕获双向上下文**的基于**自回归**的语言建模方法
- ❑ 引入了自编码语言模型中的**双向上下文**，能够利用更加丰富的上下文信息
 - ❑ 传统的自回归语言模型只能利用**单向**的历史信息
- ❑ 解决了BERT中存在的“**预训练-精调**”不一致的问题
 - ❑ 自编码语言模型中引入人造标记[MASK]的问题，“预训练-精调”不一致
 - ❑ 使用了**自回归**语言模型结构，使得各个单词的预测存在依赖
- ❑ 使用了Transformer-XL作为主体框架，相比传统的Transformer拥有更好的性能

模型优化——XLNet(2/13)

□ Transformer-XL 的核心改进

- **循环机制**：跨片段传递隐藏状态，解决上下文碎片化
- **相对位置编码**：支持任意长度序列，提升泛化能力
- **内存优化**：减少重复计算，提升长序列处理效率

□ 适用场景：

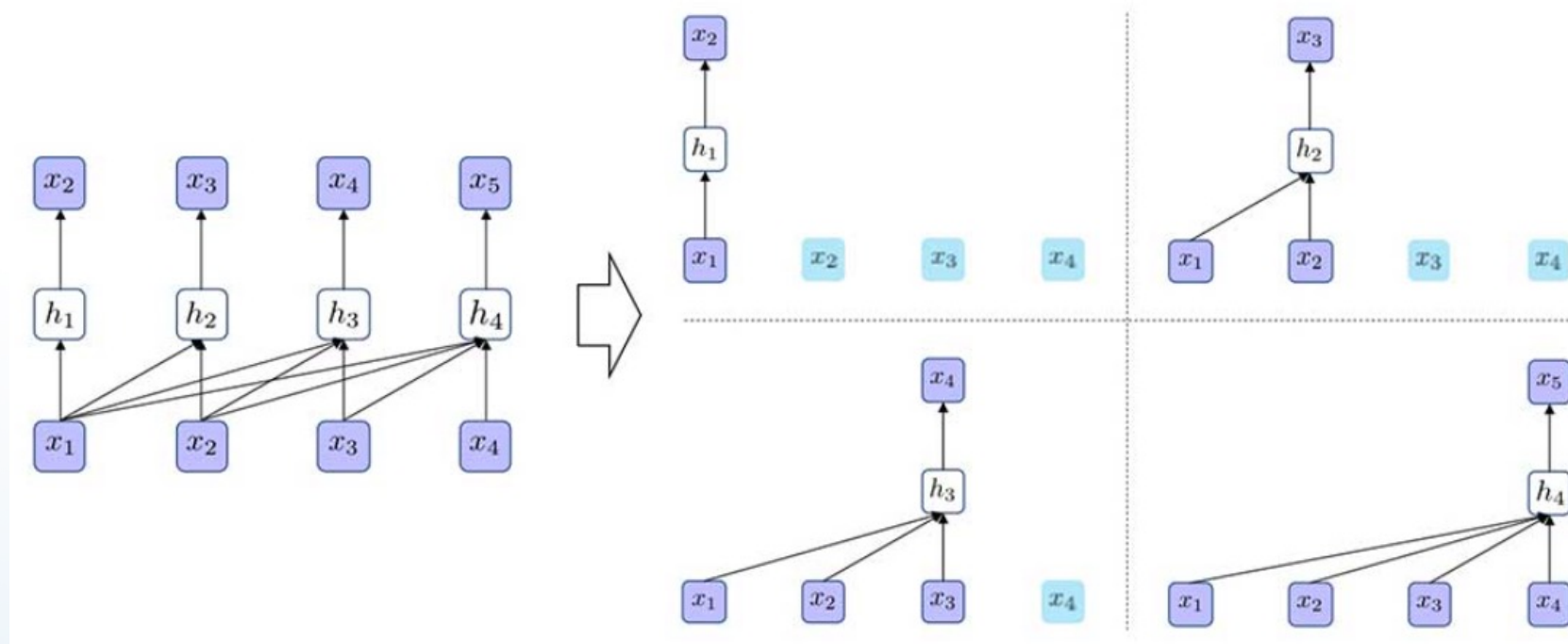
- 长文本生成(如故事、对话)
- 语言模型预训练(如 XLNet)
- 需要建模长期依赖的任务(如文档摘要)

模型优化——XLNet (3/13)

- **XLNet:** XLNet的主体结构仍然是一个自回归语言模型
 - 如何将双向上下文引入自回归语言模型？

模型优化——XLNet (4/13)

传统的自回归语言模型的示例



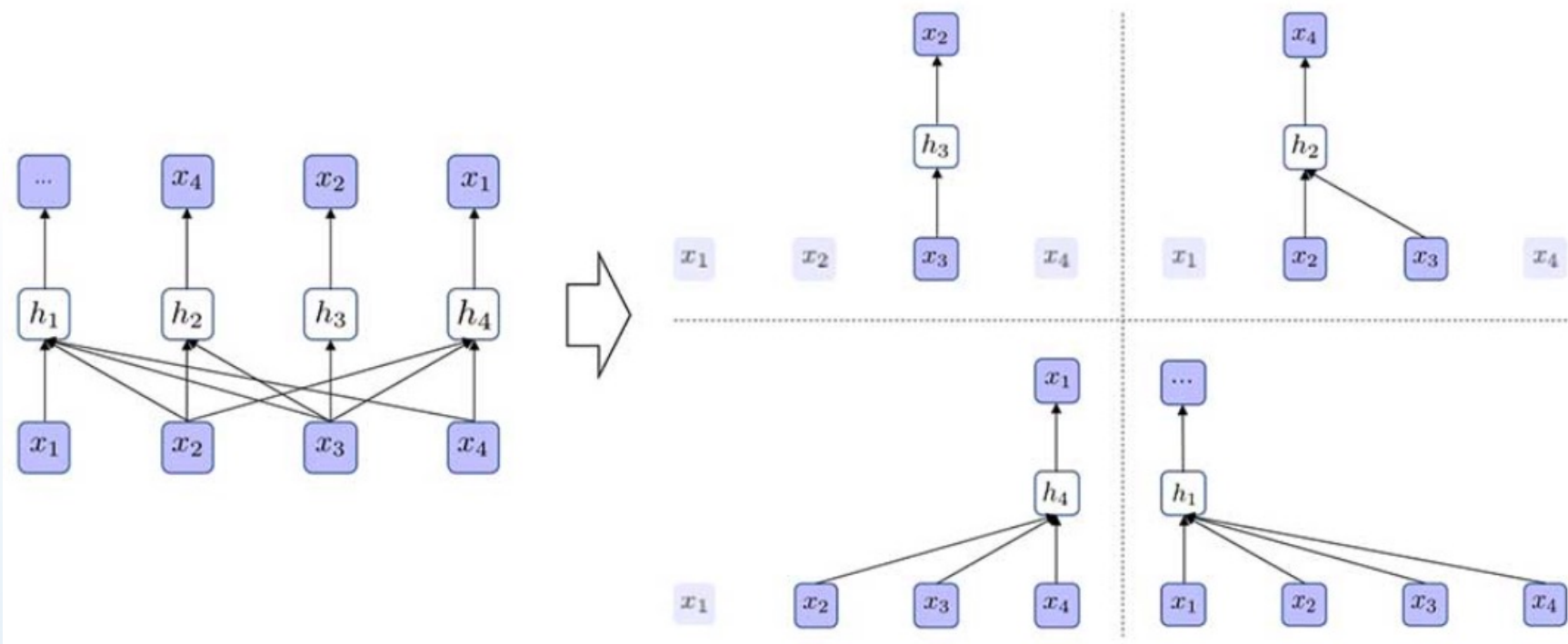
假设句子 $x = x_1 x_2 x_3 x_4$ 包含四个词，每个词只依赖其历史词，不能利用未来词进行预测。即对于单向语言模型，句子的建模顺序为：
 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

整个句子的建模为： $P(x) = P(x_1)P(x_2|x_1)P(x_3|x_2x_1)P(x_4|x_3x_2x_1)$

模型优化——XLNet (5/13)

❑ 为了构建双向上下文，XLNet提出排列语言模型

❑ 假设建模句子 $x = x_1x_2x_3x_4$ ，将建模顺序改为: $3 \rightarrow 2 \rightarrow 4 \rightarrow 1$



❑ 整个句子建模为: $P(x) = P(x_3)P(x_2|x_3) P(x_4|x_3x_2) P(x_1|x_3x_2x_4)$

- ❑ 当预测 x_4 时，需要依赖 x_2 和 x_3
- ❑ 当预测 x_1 时，需要依赖 x_2, x_3 和 x_4
- ❑ 即实现了双向上下文的建模方式

模型优化——XLNet (6/13)

排列语言模型 (Permutation Language Model)

对于给定长度为N的句子 $x = x_1 \cdots x_N$, 从所有可能的排列方式 $\mathbb{Z}_N$ 当中均匀采样一种排序 z , 最大化对数似然函数

$$\mathbb{E}_{z \sim \mathbb{Z}_N} [\log P(x|z)] = \mathbb{E}_{z \sim \mathbb{Z}_N} \sum_{i=1}^N \log P(x_{z_i} | x_{1:i-1}, z_i) \quad (8-5)$$

z_i 表示在排序 z 下 , 下一个预测词 x_{z_i} 在句子中的下标

概率分布 $P(x_{z_i} | x_{1:i-1}, z_i)$ 必须依赖于目标位置 z_i , 如何建模 ?

假设通过标准的Softmax函数建模

$$P(x_{z_i} = x | x_{1:i-1}) = \frac{\exp(v_x^T h_{x_{1:i-1}})}{\sum_{x'} \exp(v_{x'}^T h_{x_{1:i-1}})} \quad (8-6)$$

v_x : 词 x 对应的词向量, $h_{x_{1:i-1}}$: $x_{1:i-1}$ 对应的隐含层表示

$h_{x_{1:i-1}}$ 不依赖于目标位置 z_i , 式(8-6)总会产生一样的概率分布 , 这将无法满足式(8-5)的建模要求

模型优化——XLNet (7/13)

□ **XLNet:** XLNet 对式 (8-6)进行了变动，使其依赖于目标位置 z_i

$$P(x_{z_i} = x | x_{1:i-1}) = \frac{\exp(v_x^T g(x_{1:i-1}, z_i))}{\sum_{x'} \exp(v_{x'}^T g(x_{1:i-1}, z_i))} \quad (8-7)$$

□ 函数 g : 表示一种依赖于目标位置 z_i 的隐含层表示方法

□ 如何构造这样一种函数 g 呢？

① 当预测 x_{z_i} 时，函数 g 应该只需利用位置信息 z_i ，而非具体单词 x_{z_i}

□ 若函数 g 已经知道单词 x_{z_i} 是什么词，则只需直接输出 x_{z_i} 即可，从而使得函数 g 变得异常简单，无法达成语言建模的目的

② 当预测 $x_{z_j} (j > i)$ 时，函数 g 需要编码单词 x_{z_i} 以提供完整的上下文信息

□ 上述两点存在一定矛盾，无法通过同一个函数涵盖两种不同情况。

双流自注意力机制！

模型优化——XLNet (8/13)

▣ **XLNet**: 在双流自注意力机制中，同一个单词具有以下两种不同的表示

▣ **内容表示**(Content Representation) h_{z_i} ：即原始的Transformer表示方法，可以同时建模单词 x_{z_i} 及其上下文

▣ **查询表示**(Query Representation) g_{z_i} ：能建模上下文信息 $x_{z_{1:i-1}}$ 以及目标位置 z_i ，但不能看到单词 x_{z_i}

模型优化——XLNet (9/13)

□ 双流自注意力

□ 对于单词 x_{z_i} :

□ 第0层内容表示(即第1层Transformer的输入): $h_{z_i}^{[0]} = v_{x_{z_i}}$

□ $v_{x_{z_i}}$ 为输入表示

□ 第0层查询表示: 由随机初始化的可训练向量构成 $g_{z_i}^{[0]} = w$

模型优化——XLNet (10/13)

□ 双流自注意力

□ 对于单词 x_{z_i} : 对于 $l \in \{1, \dots, L\}$ 层的Transformer, 采用下

式更新内容表示 $h_{z_i}^{[l]}$ 和查询表示 $g_{z_i}^{[l]}$

$$h_{z_i}^{[l]} \leftarrow \text{Transformer-Block}(Q = h_{z_i}^{[l-1]}, K = h_{z_{\leq i}}^{[l-1]}, V = h_{z_{\leq i}}^{[l-1]}; \theta) \quad (8-8)$$

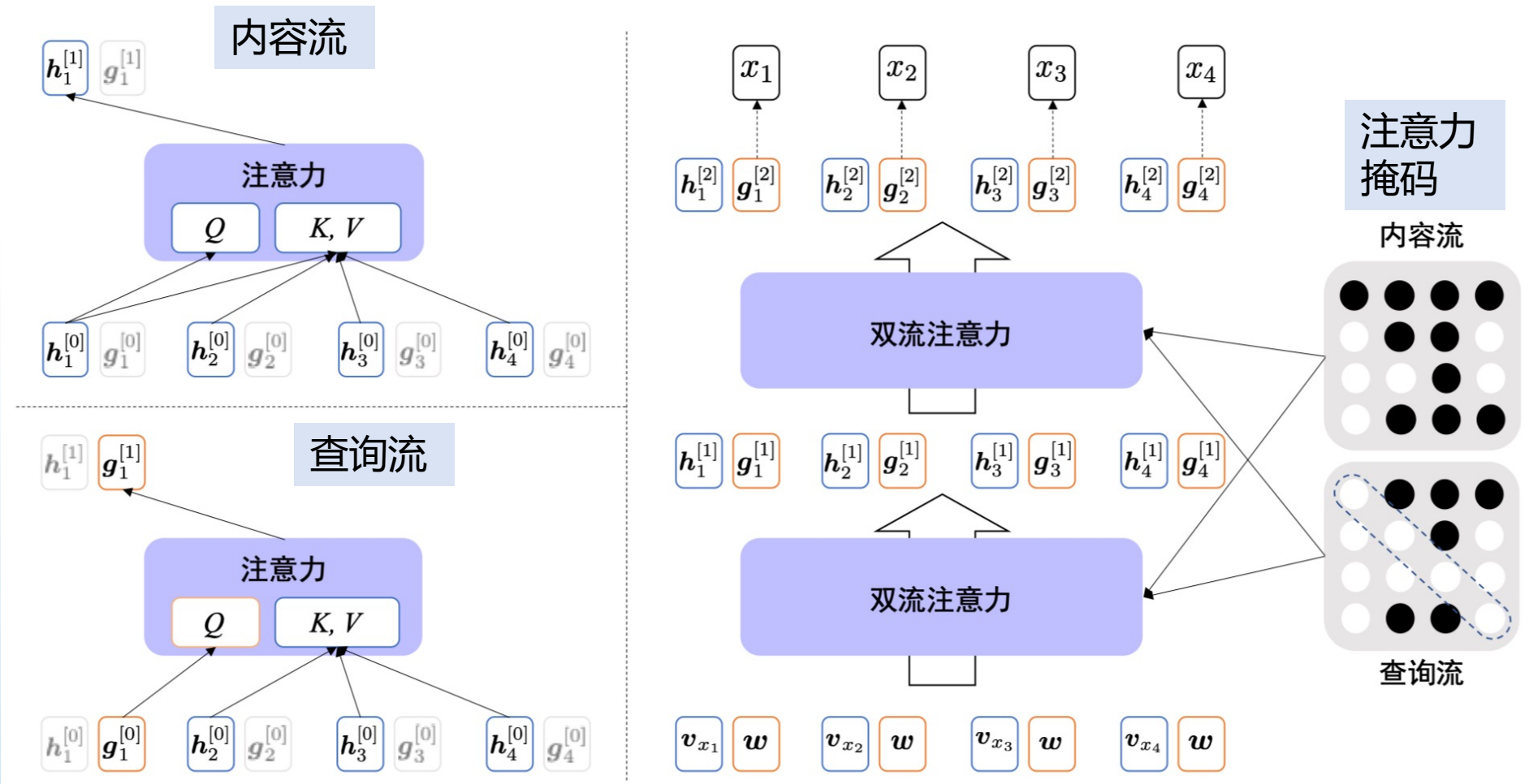
$$g_{z_i}^{[l]} \leftarrow \text{Transformer-Block}(Q = g_{z_i}^{[l-1]}, K = h_{z_{1:i-1}}^{[l-1]}, V = h_{z_{1:i-1}}^{[l-1]}; \theta) \quad (8-9)$$

□ Q、K、V 分别表示多头自注意力机制中的查询(Query)、键 (Key) 和值(Value)

□ 参数更新策略与标准的多头自注意力机制完全相同

简化了两层的XLNet模型

输入由四个词构成，即 x_1 、 x_2 、 x_3 、 x_4



模型优化——XLNet (11/13)

- ❑ 双流自注意力机制主要通过改变注意力掩码矩阵实现
- ❑ 注意力掩码矩阵:
 - ❑ 是一个方阵，代表了第 i 个词与第 j 个词之间是否存在联系
 - ❑ $M_{i,j}^h$: 对应的内容流注意力掩码矩阵
 - ❑ $M_{i,j}^g$: 查询流注意力掩码矩阵
- ❑ 对于两种不同表示的注意力掩码矩阵是不同的

模型优化——XLNet (12/13)

假设句子建模顺序为 $3 \rightarrow 2 \rightarrow 4 \rightarrow 1$, 内容流与查询流注意力掩码矩阵的构造方式如下

当前预测	当前历史	内容流置一元素	查询流置一元素
$3 \rightarrow 2 \rightarrow 4 \rightarrow 1$	$\emptyset$	$M_{3,3}^h$	$\emptyset$
$3 \rightarrow 2 \rightarrow 4 \rightarrow 1$	x_3	$M_{2,3}^h, M_{2,2}^h$	$M_{2,3}^g$
$3 \rightarrow 2 \rightarrow 4 \rightarrow 1$	x_3, x_2	$M_{4,3}^h, M_{4,2}^h, M_{4,4}^h$	$M_{4,3}^g, M_{4,2}^g$
$3 \rightarrow 2 \rightarrow 4 \rightarrow 1$	x_3, x_2, x_4	$M_{1,3}^h, M_{1,2}^h, M_{1,4}^h, M_{1,1}^h$	$M_{1,3}^g, M_{1,2}^g, M_{1,4}^g$



两者的区别: 掩码矩阵的对角线是否为0, 即当前词能否看到自己本身的内容

内容流: 能看自己本身的内容

查询流: 不能看到自己本身的内容

模型优化——XLNet (13/13)

- 最后，使用第L层(即最后一层)Transformer的查询表示 $g_{z_i}^{[L]}$ 计算式(8-7)

$$P(x_{z_i} = x | x_{1:i-1}) = \frac{\exp(v_x^T g(x_{1:i-1}, z_i))}{\sum_{x'} \exp(v_{x'}^T g(x_{1:i-1}, z_i))} \quad (8-7)$$

- 注意: 查询表示是排列语言模型预训练任务额外引入的，在精调下游任务时并不会使用
- 除了排列语言模型和双流自注意力机制，XLNet还
 - 引入部分预测进一步提升排列语言模型的收敛速度
 - 使用相对块编码技术提升模型对不同输入形式的泛化能力
 - 相关技术的实现方法见文献[27]

模型优化—— RoBERTa (1/13)

□ RoBERTa

- 训练BERT需要耗费大量的数据资源和计算资源，比较不同的模型设计决策变得非常困难
- 为了进一步了解 BERT 的设计合理性，Liu 等人提出了 RoBERTa^[29]，通过大量的实验表明BERT的设计仍然存在较大的改进空间
- RoBERTa模型没有大刀阔斧地调整BERT，针对每一个设计细节做了详尽的实验，并通过实证方法进一步优化了BERT，并在一系列NLP任务中取得了当时最好的效果
- RoBERTa在BERT的基础上
 - 引入了动态掩码技术
 - 舍弃了NSP任务
 - RoBERTa采用了更大规模的预训练数据，并以更大的批次和BPE词表训练了更多的步数

模型优化—— RoBERTa (2/13)

□ 静态掩码 vs. 动态掩码

- BERT中的MLM任务会对输入文本中的部分单词随机掩码
- 掩码过程是在数据预处理阶段进行的，而非模型训练阶段
- 导致生成的掩码是静态的：同一个文本只有一种掩码模式，降低了训练数据的多样性以及数据的复用效率
- 为了缓解这个问题，在BERT的原始实现中，将训练数据复制了10份。对于同一个文本就会生成10种不同的掩码模式。但BERT的总训练轮数是40轮左右，同一个掩码模式仍会重复4次。
- 动态掩码：模型训练的**过程中**决定哪些词被mask

模型优化—— RoBERTa (3/13)

□ 1 动态掩码

□ 动态掩码：在模型的训练阶段实时计算掩码位置和方法

轮数 ↓	went to the store → went to the [MASK]	went to the store → went to the [MASK]
	went to the store → went to the [MASK]	went to the store → went to [MASK] store
	went to the store → went to the [MASK]	went to the store → go to the store
	went to the store → went to the [MASK]	went to the store → went to the store
	went to the store → went to the [MASK]	went to the store → went [MASK] the store
静态掩码		动态掩码

□ 优点：

- 无论训练多少轮，都能够最大限度地保证同一段文本能够在不同轮数下产生不同的掩码模式
- 当预训练轮数较大或数据量较大时，动态掩码方法能够提高数据的复用效率

模型优化—— RoBERTa (4/13)

□1动态掩码

□实验结果：使用动态掩码技术的BERT

- ① 在阅读理解数据集SQuAD 2.0^[30]以及文本分类数据集SST-2^[20]任务上，能够带来微弱的性能提升
- ② 在MNLI-m^[31]任务上有一定的性能损失

掩码方法	SQuAD 2.0	MNLI-m	SST-2
静态掩码	78.3	84.3	92.5
动态掩码	78.7	84.0	92.9

❑ 2舍弃NSP任务

- ❑ NSP预训练任务是否是必要的？

- ❑ 在原始BERT的分析实验中，去掉NSP任务会显著降低QNLI^[32](自然语言推断)任务、MNLI^[31](自然语言推断)任务和SQuAD 1.1^[22](阅读理解)任务的效果。
- ❑ 为了更好地了解NSP任务的有效性，RoBERTa论文作者对比了以下4种实验设置。

模型优化—— RoBERTa (6/13)

❑ 2 舍弃NSP任务

❑ RoBERTa论文作者对比了以下4种实验设置

- ① 文本对输入+NSP：原始BERT的输入形式，即由一对文本构成，每个文本由多个自然句子组成，整体长度不超过512个标记；
- ② 句子对输入+NSP: 由一对句子构成输入序列
 - ❑ 大多数情况下，一对句子的长度小于512，通过增大批次大小保持和“文本对输入”相对一致的数据吞吐量；
- ③ 跨文档整句输入:由一对文本构成输入序列。当达到文档的末端时，将继续从下一个文档抽取句子，并添加分割符表示文档边界。不再使用NSP损失
- ④ 文档内整句输入:与“跨文档整句输入”类似，但当达到文档末端时，不允许继续从下个文档中抽取句子。
 - ❑ 通过增大批次大小保持和“跨文档整句输入”相对一致的数据吞吐量。不再使用NSP损失。

❑ 2舍弃NSP任务

❑ 实验结果

实验设置	SQuAD 1.1	SQuAD 2.0	MNLI-m	SST-2	RACE
文本对输入 + NSP	90.4	78.7	84.0	92.9	64.2
句子对输入 + NSP	88.7	76.2	82.9	92.1	63.0
跨文档整句输入	90.4	79.1	84.7	92.5	64.8
文档内整句输入	90.6	79.7	84.7	92.7	65.6

❑ 使用NSP任务的情况下，只使用“句子对输入”相比使用“文本对输入”带来一定的性能损失。

❑ 原因：“句子对输入”的**长度较短**，无法学习到**长距离依赖**，对阅读理解任务SQuAD 1.1以及RACE^[33]等需要长距离理解的任务带来较大的影响。

❑ 使用NSP任务(前两行)vs不使用NSP任务(后两行):不使用NSP任务能够带来下游任务(SST-2(情感分类)除外)的性能提升

❑ “跨文档整句输入” vs “文档内整句输入” :后者的实验效果更好

❑ 使用“文档内整句输入”的模式会导致**批次大小**是一个**可变量**，对于大规模预训练并不友好。

❑ RoBERTa最后采用了“**跨文档整句输入**”并舍弃了NSP任务的方案。29

模型优化—— RoBERTa (7/ 13)

□3 其他优化

□RoBERTa还引入了更多的预训练数据、使用了更大的批次、更长的预训练步数和更大的BPE词表

□(1)更多的预训练数据。

□在原始BERT中，预训练数据采用的是BookCorpus和英文维基百科数据，总计约16GB的文本文件。

□RoBERTa进一步将预训练数据的规模扩展至160GB，是BERT的10倍。

□RoBERTa的预训练数据共包含5个数据来源，相关描述如下

数据名称	文本类型	大小
BookCorpus	故事	16 GB
Wikipedia	百科	
CC-News	新闻	76 GB
OpenWebText	社区问答	38 GB
Stories	故事	31 GB

模型优化—— RoBERTa (8/ 13)

□3 其他优化

- (2)更大的批次及更长的预训练步数。在原始BERT中，采用的预训练批次大小为256，并训练了1M步。
- 在RoBERTa中，进一步探索了更大的批次以及更长的训练步数能否带来进一步性能提升。相关结果如下

批次大小	训练步数	PPL	MNLI-m	SST-2
256	1M	3.99	84.7	92.5
	125K	3.68	85.2	93.1
2,048 (2K)	250K	3.59	85.3	94.1
	500K	3.51	85.4	93.5
8,192 (8K)	31K	3.77	84.4	93.2
	63K	3.60	85.3	93.5
	125K	3.50	85.8	94.1

模型优化—— RoBERTa (9/ 13)

□3 其他优化

批次大小	训练步数	PPL	MNLI-m	SST-2
256	1M	3.99	84.7	92.5
	125K	3.68	85.2	93.1
2,048 (2K)	250K	3.59	85.3	94.1
	500K	3.51	85.4	93.5
8,192 (8K)	31K	3.77	84.4	93.2
	63K	3.60	85.3	93.5
	125K	3.50	85.8	94.1

- 随着批次大小的增大，不论是在开发集上的困惑度(PPL)还是在实际的下游任务(MNLI-m、SST-2)上均有一定的性能提升。
 - 由于预训练通常需要花费很多时间，在计算资源充裕的情况下，使用更大的批次能够有效减少训练时长。
- 当固定批次大小并增加训练步数后，也能得到更好的实验结果
- 基于以上实验结果，最终RoBERTa采用了8K的批次大小，并且进一步将训练步长加大至500K。

模型优化—— RoBERTa (10/ 13)

□3 其他优化

□(3)更大的词表。在原始BERT中，采用了一个30K大小的WordPiece^[34]词表

□特点：一种基于字符级别(Char-level)的BPE^[35]词表。

□弊端：如果输入文本无法通过词表中的WordPiece子词进行拼接组合，则会映射到“unknown”这种未登录词标识

□RoBERTa模型使用了SentencePiece分词器，并且将词表大小扩大至50K

□特点：字节级别(Byte-level) BPE词表

□优点：能够编码任意输入文本，因此不会出现未登录词的情况

模型优化—— RoBERTa (11/ 13)

□3 其他优化

□例如

- 这里使用英文BERT和RoBERTa词表对输入文本进行分词
- 输入文本中包含英文、**德文**、**中文**和**日文**

```
# 加载BERT和RoBERTa分词器，并设置未登录词以 '[UNK]' 显示。
>>> from transformers import BertTokenizer, RobertaTokenizer
>>> bert_tokenizer = BertTokenizer.from_pretrained('bert-base-uncased',
    unk_token=' [UNK] ')
>>> roberta_tokenizer = RobertaTokenizer.from_pretrained('roberta-base',
    unk_token=' [UNK] ')
# 由4种语言组成的输入文本列表。
>>> sents = ['Harbin Institute of Technology', 'Harbin Institut für
    Technologie', '哈尔滨工业大学', 'ハルビン工業大学']
```

模型优化—— RoBERTa (12/ 13)

□3 其他优化

□应用BERT中的分词器进行分词，其结果如下所示：

```
>>> [bert_tokenizer.tokenize(x) for x in sents]
[['ha', '##rbin', 'institute', 'of', 'technology'],
 ['ha', '##rbin', 'institut', 'fur', 'techno', '##logie'],
 ['[UNK]', '[UNK]', '[UNK]', '[UNK]', '[UNK]', '大', '学'],
 ['ハ', '##ル', '##ヒ', '##ン', '[UNK]', '[UNK]', '大', '学']]
```

- 拉丁语系的英文和德文的分词结果均未出现未登录词的情况
- 对于中文和日文的部分,词汇出现词表中无法映射的未登录词

模型优化—— RoBERTa (13/ 13)

□3 其他优化

□应用RoBERTa中的分词器进行分词，其结果如下所示。

```
>>> segs_list = [roberta_tokenizer.tokenize(x) for x in sents]
>>> ['[UNK]' in x for x in segs_list]
[False, False, False, False]
```

□SentencePiece是字节级别的切分，部分单词在切分后不可读(打印出来呈乱码)，这里直接通过判断的形式查看列表中是否包含未登录词。

□**结果**：列表中所有元素均不包含 “[UNK]”，说明所有单词均被正常映射。

模型优化—— ALBERT (1/11)

▣ALBERT

▣Bert模型

▣**优点**：在众多自然语言处理任务中取得显著的性能提升

▣**问题**：参数量相对较大，会占用大量计算资源

▣Lan等人提出了ALBERT(A Lite BERT)^[36]**降低内存的消耗**
并且**提高BERT的训练速度**

▣更加小巧(从参数量的角度)的预训练

▣两种主要技术：

① 词向量因式分解

② 跨层参数共享

③ 引入了更加有效的“**句子顺序预测**”的预训练任务，取代了BERT中原有的NSP任务

模型优化——ALBERT (2/11)

□ALBERT

□(1)词向量因式分解

□BERT以及相关变种模型(如XLNet、RoBERTa等)中，词向量的维度E和Transformer的隐含层维度H是一样的。这种设计决策存在两个问题。

- ① 从模型设计角度来看，词向量的作用是将输入文本映射到上下文无关的静态表示中，即输入文本中的每个标记会独立地通过词向量矩阵映射到一个固定的向量，与其上下文无关。而大量的实验表明，以BERT为代表的预训练语言模型之所以强大，是因为词向量之上建立的深层Transformer模型能够充分地学习到每个标记的上下文信息。因此，ALBERT的作者认为，Transformer的隐含层维度H要远大于词向量维度E，即 $H \gg E$ 。

模型优化——ALBERT (3/11)

□ALBERT

□(1)词向量因式分解

② 从实用角度来看，词向量矩阵的参数量是词表大小 V 乘以词向量维度 E

□在通常情况下,词表大小 V 是比较大的。例如，BERT的词表大小是30K

□上文提到，在早期的预训练语言模型的设计中 $H \equiv E$ 。当通过增大 H 提升模型容量时，词向量维度 E 也随之增大，词向量矩阵的参数量也会随之上升

□另外，词向量矩阵的更新是比较稀疏的，参数的利用率并不高

□ALBERT引入词向量因式分解解耦合词向量维度 E 和隐含层维度 H

模型优化——ALBERT (4/11)

▣ALBERT

▣(1)词向量因式分解

▣ALBERT引入词向量因式分解解耦合词向量维度 E 和隐含层维度

H ，具体的操作方法令 $H \neq E$

- ▣当 $H \neq E$ 时，词向量不能直接接入后续的多层Transformer模型中

- ▣需引入一个全连接层，将词向量维度 E 映射到Transformer隐含层维度 H

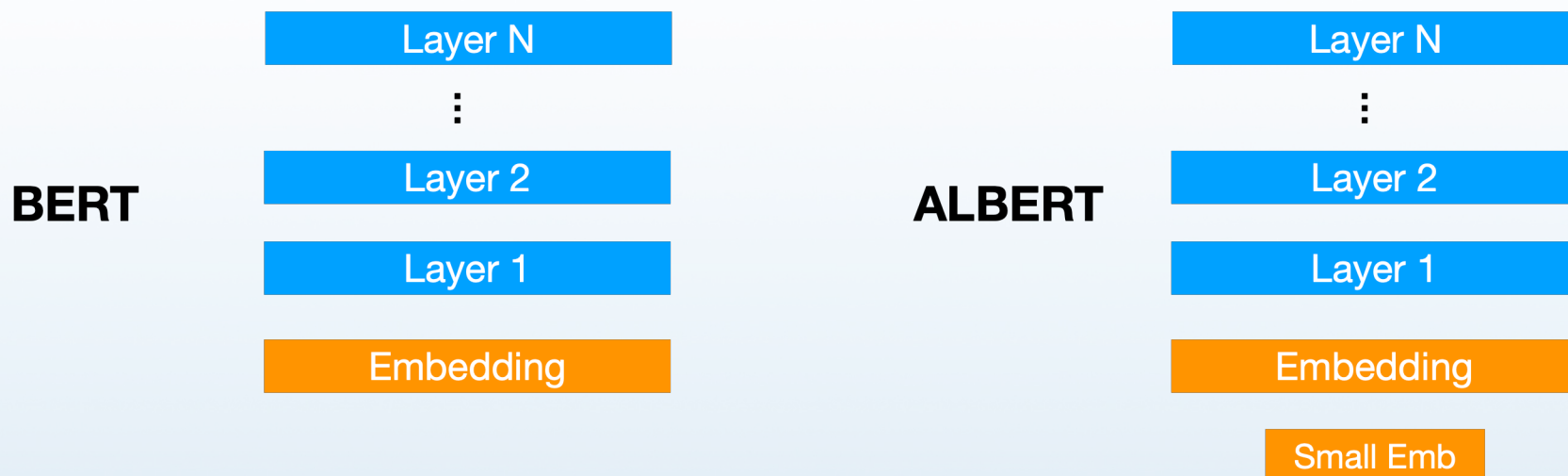
- ▣词向量部分的计算复杂度从 $O(VH)$ 降低至 $O(VE + EH)$

- ▣当Transformer隐含层维度 H 远大于词向量维度 E 时，参数量的降幅尤为明显

模型优化—— ALBERT (5/11)

词向量因式分解 (Factorized Embedding Parameterization)

- 在BERT中, $\text{embedding_size} == \text{hidden_size}$
- 在ALBERT中, $\text{embedding_size} < \text{hidden_size}$



模型优化——ALBERT (6/11)

□ALBERT

□利用词向量因式分解后

$$O(V \times H) \quad \longrightarrow \quad O(V \times E + E \times H)$$

□举例： $V=30000, H=2014, E=128$

BERT

>

ALBERT

$$V * H = 30000 * 1024 = 30,720,000$$

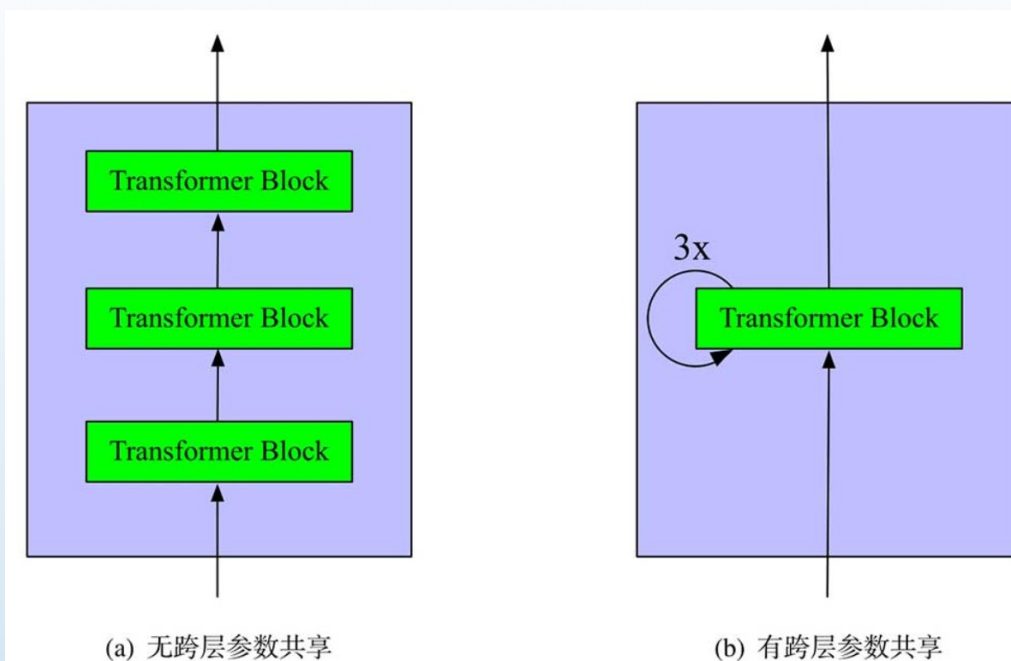
$$V * E + E * H = 30000 * 128 + 128 * 1024 = 3,971,072$$

模型优化——ALBERT (7/11)

ALBERT

(2)跨层参数共享

- 在BERT中，多层Transformer的参数是不共享的，即每一层Transformer都保留自己的参数
- ALBERT引入了跨层参数共享机制，使得每一层Transformer的权重都是一样的。三层Transformer模型说明跨层参数共享



模型优化—— ALBERT (8/11)

□ALBERT

□(2)跨层参数共享

- ALBERT采用了一种类似于“循环”的结构，主体结构部分实际上只包含一层Transformer实体。
- 通过循环操作，Transformer的参数得到复用，并且可以实现深层计算(即循环多少次就是多少层)。
- 注意：跨层参数共享虽然从参数量的角度实现了模型的压缩，但并不会加快模型的前向计算时间，也不会大幅度减少模型的内存(或显存)占用。
- 三层Transformer模型为例，规定每层的基准参数量、磁盘占用、内存占用和前向传播时间为1x，

模型优化——ALBERT (9/11)

ALBERT

(2)跨层参数共享

三层Transformer模型为例，规定每层的基准参数量、磁盘占用、内存占用和前向传播时间为1x

对比项目	参数量	磁盘占用	内存占用	前向传播时间
无跨层参数共享（3层）	3x	3x	3x	3x
有跨层参数共享（3层）	1x	1x	3x	3x

参数量的大小直接影响磁盘占用，更少的参数量可用更小的文件存储

内存占用、前向传播时间与有无跨层参数共享无关

不论在模型训练还是模型推断时，共享的参数仍然要以虚拟的形式复制成多份，形成多层Transformer结构，内存的占用并没有减少

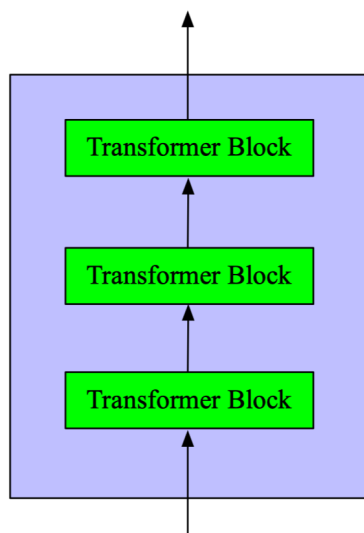
模型的输入还是要从Transformer的最底层一步步地传递到Transformer的最顶层，因此前向传播时间并没有什么变化

模型优化——ALBERT (10/11)

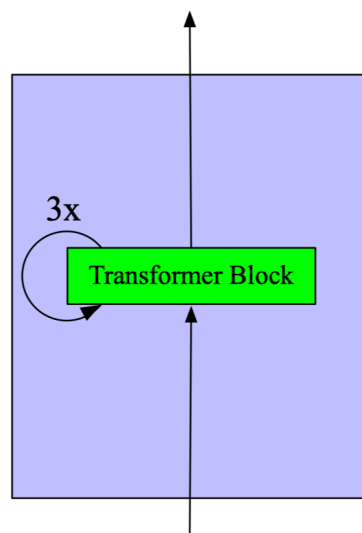
□ALBERT

□(2)跨层参数共享

- Transformer每层的参数是**共享**的，即只需要存一份参数，与层数无关
- 训练时**：虽然参数共享，但每层的梯度是不同的，仍然需要额外空间存储
- 推断时**：前向计算过程仍然要一层层展开，**并不能节省推断时间**



(a) 无跨层参数共享



(b) 有跨层参数共享

模型优化—— ALBERT (11/11)

□ALBERT

□(3)句子顺序预测 (Sentence Order Prediction, SOP)

□NSP

□任务实际隐含了“连贯性”和“主题”预测

□判断两段文本的主题是否一致是比较容易的

□ALBERT提出句子顺序预测(Sentence Order Prediction , SOP)任务

□**优势**：学习到**细微的语义差别**及**语篇连贯性**,比NSP任务难度更大

□**样本**：

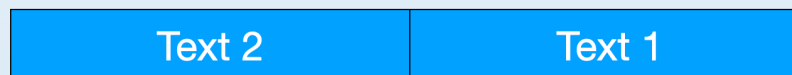
□**正样本**：与BERT相同，由两个连续的文本段组成

+



□**负样本**：交换两个连续文本段的顺序

—



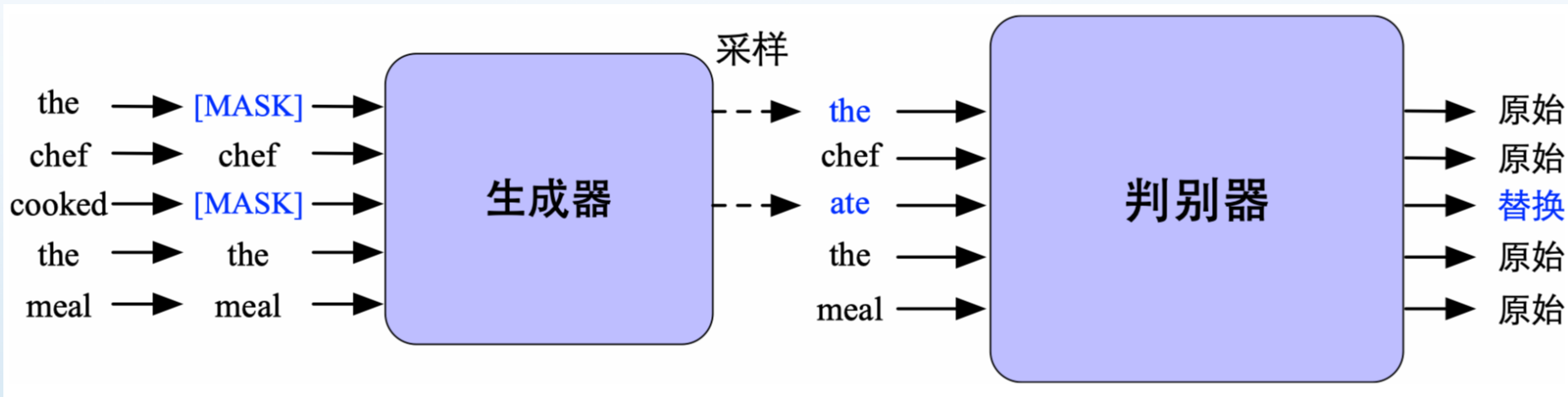
模型优化—— ELECTRA (1/9)

□ ELECTRA

- 提出了一种全新基于生成器-判别器框架的预训练语言模型
- 相比传统预训练模型的训练效率更高

□ 整体结构

- 生成器-判别器框架，与GAN (Goodfellow et al., 2014) 类似
- **Generator**: 将输入中的缺失信息还原为原单词
- **Discriminator**: 判断输入的单词是否被替换过



模型优化—— ELECTRA (2/9)

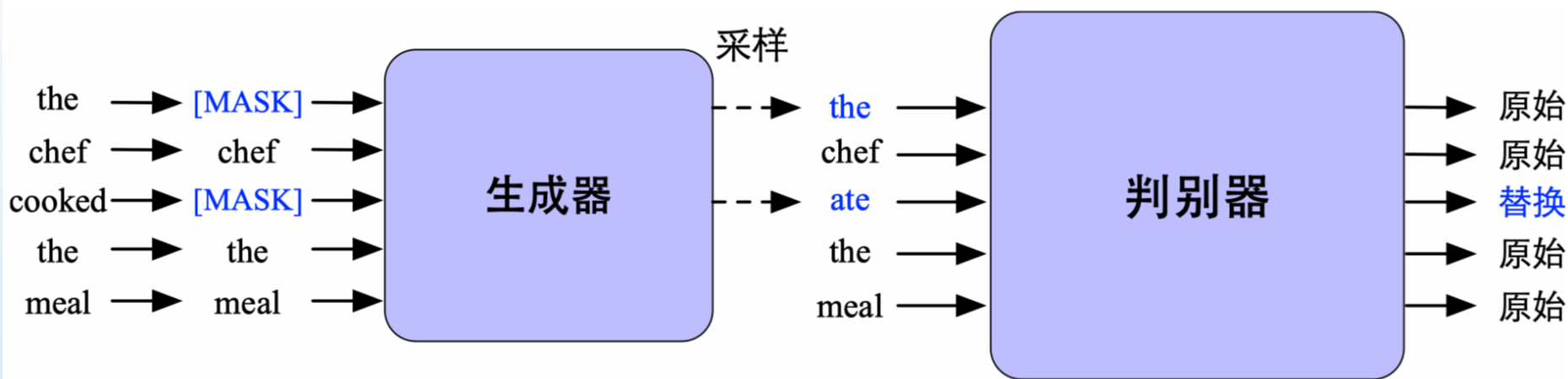
□ ELECTRA

□ 生成器：一个小的MLM

- 第一步：随机选取输入序列中的一部分进行遮蔽，通常比例选取15%
- 第二步：利用MLM任务，将缺失信息还原为原单词

□ 判别器：常规的BERT结构

- 第一步：将缺失信息替换为生成器预测出来的单词
- 第二步：判别器学习判断输入句子中哪些单词被生成器替换过



模型优化—— ELECTRA (3/9)

□ ELECTRA

□ 生成器

□ **目的**: 将带有掩码的输入文本 $x = x_1 x_2 \cdots x_n$, 通过多层 Transformer 模型, 学习到上下文语义表示 $h = h_1 h_2 \cdots h_n$, 并还原掩码位置的文本, 即 BERT 中的 MLM 任务

□ **注意**: 只预测经过掩码的词, 即对于某个掩码位置 t , 生成器输出对应原文本 x_t 的概率 $P^G \in R^{|V|}$ ($|V|$ 是词表大小):

$$P^G(x_t|x) = \text{softmax}(h_t^G W^e{}^T)$$

□ $W^e \in R^{|V| \times d}$ 表示词向量矩阵; h_t^G 表示原文本 x_t 对应的隐含层表示。

模型优化—— ELECTRA (4/9)

□ ELECTRA

□ 生成器

□ 例 原始句子 $x = x_1x_2x_3x_4x_5$ 如下：

□ the chef cooked the meal

□ 经过随机掩码后的句子如下

□ [MASK] chef [MASK] the meal

□ 记 $M = \{1,3\}$ 为所有经过掩码的单词位置的下标，记 $x^m = x_1x_2x_3x_4x_5$ 为经过掩码后的输入句子

□ 生成器的目标: 将 m_1 还原为 x_1 (即 the), 将 m_3 还原为 x_3 (即 cooked)。

模型优化—— ELECTRA (5/9)

□ ELECTRA

□ 生成器

- 在理想情况下，即当生成器的准确率为100%时，掩码标记[MASK]能够准确还原为原始句子中的对应单词
- 在实际情况下，MLM的准确率并没有那么高。如将掩码后的句子 x^m 输入生成器中，将产生采样后的句子 x^s ：
 - the chef ate the meal
 - m_1 通过生成器成功地还原出单词the
 - m_3 采样(或预测)出的单词是ate，不是cooked
- 生成器生成的句子将会作为判别器的输入
 - 通过生成器改写后的句子中不包含任何人为预先设置的符号(如[MASK])，解决了预训练和下游任务输入不一致的问题

模型优化—— ELECTRA (6/9)

□ ELECTRA

□ 判别器

□ 受MLM准确率的影响，通过生成器采样后的句子 x^s 与原始句子有一定的差别

□ 判别器的**目标**：从采样后的句子中识别出哪些单词是和原始句子对应位置的单词一样的，即**替换词检测**任务。

□ 可通过二分类方法实现

□ 对于给定的采样句子 x^s ，通过Transformers模型得到对应的隐含层表示 $h^D = h_1^D \cdots h_n^D$

□ 通过一个全连接层对每个时刻的隐含层表示映射成概率

$$\square P^D(h_i^s) = \sigma(h_i^D w), \quad \forall i \in M$$

□ $w \in R^d$ 表示全连接层的权重(d 表示隐含层维度)

□ M 表示所有经过掩码的单词位置下标

□ σ 表示Sigmoid激活函数

□ 假设1代表被替换过，0代表未被替换过，则生成器采样生成的句子(the chef ate the meal) 对应的预测标签为

$$y = y_1 y_2 y_3 y_4 y_5 (00100)$$

模型优化—— ELECTRA (7/9)

□ ELECTRA

□ 模型训练

□ 生成器和判别器分别使用以下损失函数训练：

$$L^G = - \sum_{i \in S} \log P^G(x_i)$$

$$L^D = - \sum_{i \in S} [y_i \log P^D(x_i^S) + (1 - y_i) \log(1 - P^D(x_i^S))]]$$

□ 模型通过最小化以下损失学习模型参数：

$$\min_{\theta^G, \theta^D} \sum_{x \in X} L^G(x, \theta^G) + \varepsilon L^D(x, \theta^D)$$

□ X : 整个大规模语料库; θ^G 和 θ^D : 生成器和判别器的参数

□ 注意：

□ 生成器和判别器衔接的部分涉及采样环节，判别器的损失并不会直接回传到生成器，采样操作不可导

□ 预训练结束后，只使用判别器进行下游任务精调，不再使用生成器

模型优化—— ELECTRA (8/9)

□ ELECTRA

□ 其他改进

□ (1) 更小的生成器

- 生成器和判别器的主体结构均由BERT组成，两者可使用同等大小的参数规模，会导致预训练的时间大约为单个模型的两倍
- 为了提高预训练的效率，在ELECTRA中生成器的参数量要小于判别器
 - 具体实现时会减小生成器中Transformer的隐含层维度、全连接层维度和注意力头的数目
 - 不同模型规模的判别器，其缩放比例也不同，通常在1/4 ~ 1/2之间
 - ELECTRA-base模型的生成器和判别器的各项参数大小对比，缩放比例是1/3

类型	词向量维度	层数	隐含层维度	全连接层维度	注意力头数	注意力头维度
生成器	768	12	256	1,024	4	64
判别器	768	12	768	3,072	12	64

- 为什么是减小生成器的大小，而非判别器的大小？
 - 生成器只会在预训练阶段使用，在下游任务精调阶段是不使用的，故减小生成器的大小是合理的

模型优化—— ELECTRA (9/9)

□ ELECTRA

□ 其他改进

□ (2) **参数共享** 为了实现更灵活的建模目的，ELECTRA

- ① 首先引入了词向量因式分解方法，通过全连接层将词向量维度映射到隐含层维度(与ALBERT中的方法一致)
- ② 在ELECTRA中，参数共享只限于输入层权重，其中包括**词向量**和**位置向量矩阵**
- ③ ELECTRA使用了一个**更小的生成器**，故生成器和判别器之间无法直接进行参数共享

模型优化—— MacBERT(1/1)

MacBERT

- 在可比条件下，评测了主流预训练模型在中文下的表现
- 提出一种新的预训练模型MacBERT，解决“预训练-精调”不一致的问题

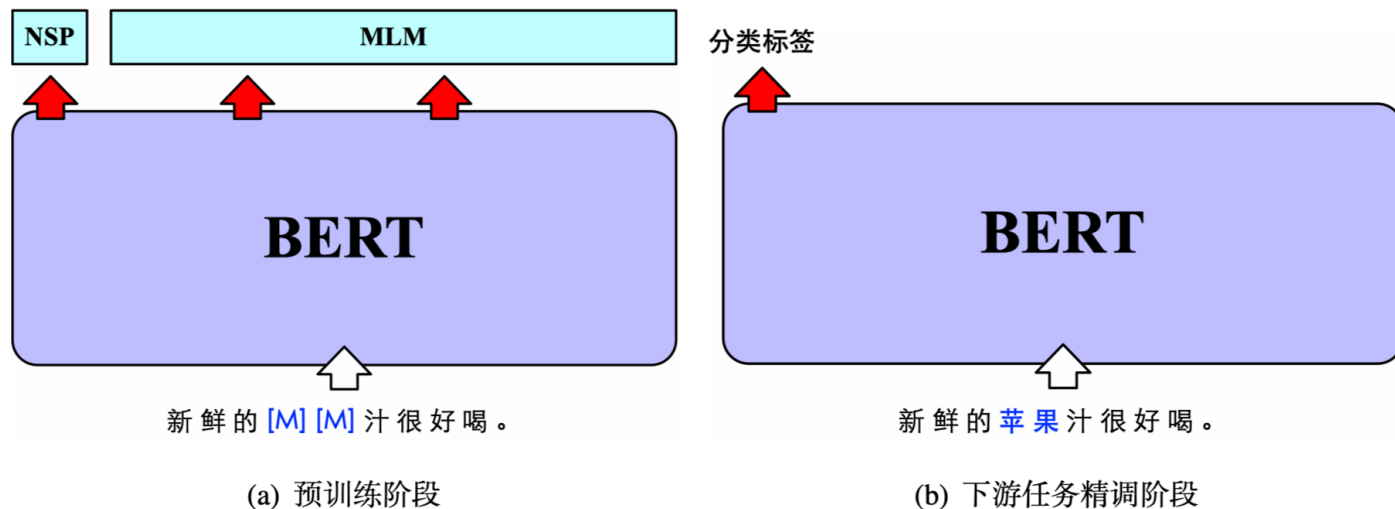


图 8-7 “预训练-精调”不一致问题示例

- 文中涉及到的所有中文预训练模型已开源至整个研究社区

MacBERT——预训练算法描述

- 使用了整词掩码和N-gram掩码，其中 unigram 至 4-gram 的概率分别为40%、30%、20% 和 10%；
- 为了解决掩码标记 [MASK] 在下游任务中不会出现的问题，在预训练阶段，MacBERT 使用相似词替换 [MASK] 标记。实际操作时，使用同义词词典获取待掩码单词的相似词。当 N-gram 掩码时，对 N-gram 中的每个词均进行相似词替换

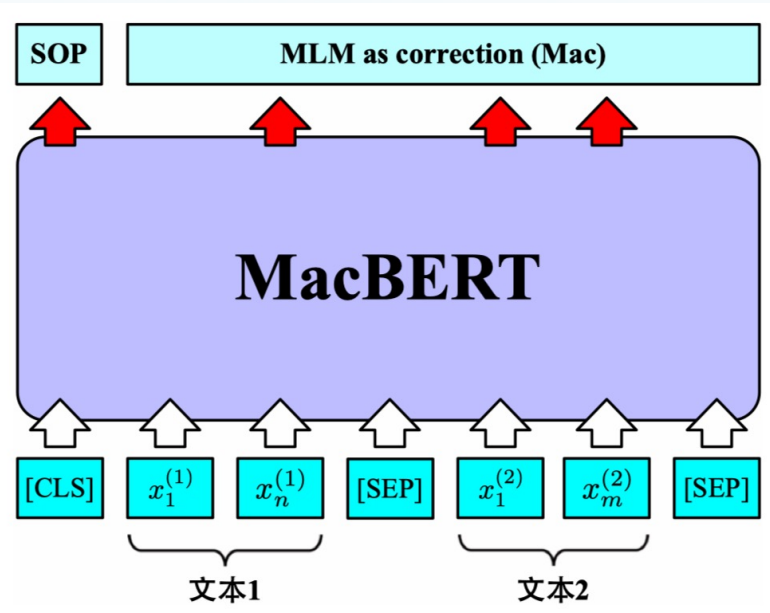


图 8-8 MacBERT 模型的整体结构

- 与原版 BERT 类似，MacBERT 对输入序列总长度 15% 的标记进行掩码，在 80% 的情况下会替换为相似词，在 10% 的情况下会替换为随机词，在 10% 的情况下则不进行任何替换(负样本)。

表 8-8 不同掩码方式的对比示例

原始句子	使用语言模型来预测下一个词的概率。
中文分词	使用语言模型来预测下一个词的概率。
原始掩码输入	使用语言[M]型来[M]测下一个词的概率。
整词掩码输入	使用语言[M][M]来[M][M]下一个词的概率。
N-gram 掩码输入	使用[M][M][M][M]来[M][M]下一个词的概率。
纠错型掩码输入	使用语法建模来预见下一个词的几率。

MacBERT (1/2)

MacBERT采用了句子顺序预测任务替换BERT中的下一个句子预测任务

ALBERT^[36]在众多自然语言处理任务上获得了显著的性能提升

预训练语言模型之间的联系与区别

模型	类型	切词	预训练任务	训练数据规模
BERT	自编码	WordPiece	MLM + NSP	≈ 16 GB
XLNet	自回归	SentencePiece	PLM	≈ 126 GB
RoBERTa	自编码	SentencePiece	MLM	≈ 160 GB
ALBERT	自编码	SentencePiece	MLM + SOP	≈ 16 GB
ELECTRA	自编码	WordPiece	Generator + Discriminator	≈ 126 GB
MacBERT	自编码	WordPiece	Mac + SOP	≈ 20 GB

Thanks!
Q&A ?

wuxianyan@njau.edu.cn